

Module 12 Lab Session 2: Frontend Vitest, Playwright E2E & Business Rule Sprint

Session 2 of 2

Exercises Exercises 4, 5, and 6

Integrity **Tier 2** Vitest Angular specs, Playwright E2E with auth storageState,
Lab tier MaxEnrollmentsPerStudent business rule

Story thread

Session 1 pinned the .NET logic and the HTTP pipeline. Session 2 turns the safety net toward the user-facing surface: Angular components and stores, full-browser user journeys, and a new business rule (MaxEnrollmentsPerStudent) added with a guiding test.

Exercise 4: Angular Component and Store Tests (Vitest)

Context: Your Angular components and SignalStores form the UI layer of the TMS. A broken component template or wrong computed signal calculation breaks the user experience even if the C# API is perfect. In this exercise, you will write Vitest specs for Angular components and NgRx SignalStores.

Part A: Component Spec (`course-card.component.spec.ts`)

CourseCardComponent (M8) uses routerLink in its template. Components that touch the router need provideRouter([]) so the ActivatedRoute and RouterLink directives resolve. Without it, the test fails with NG0201: No provider found for ActivatedRoute.

Create `src/app/ui/course-card/course-card.component.spec.ts`:

```
describe("CourseCardComponent", () => {
  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [provideRouter([])],
    });
  });

  it("should display the course title", async () => {
    const fixture = TestBed.createComponent(CourseCardComponent);
```

```

// Set signal-based required input
fixture.componentRef.setInput("course", {
  id: 1,
  code: "CSE-101",
  title: "Advanced Web Dev",
  maxCapacity: 30,
  enrollmentCount: 12,
});
await fixture.whenStable();

const el = fixture.nativeElement as HTMLElement;
expect(el.textContent).toContain("Advanced Web Dev");
});

it("should emit enrollClicked event when button is clicked", async ()
=> {
  const fixture = TestBed.createComponent(CourseCardComponent);
  const component = fixture.componentInstance;

  fixture.componentRef.setInput("course", {
    id: 1,
    code: "CSE-101",
    title: "Advanced Web Dev",
    maxCapacity: 30,
    enrollmentCount: 12,
  });
  await fixture.whenStable();

  let emittedCourse: any = null;
  component.enrollClicked.subscribe((c: any) => (emittedCourse = c));

  const button = fixture.nativeElement.querySelector(
    "button",
  ) as HTMLButtonElement;
  button.click();
  await fixture.whenStable();

  expect(emittedCourse).toBeTruthy();
  expect(emittedCourse.title).toBe("Advanced Web Dev");
});
});

```

Part B: HTTP Mock Spec (enrollment.service.spec.ts)

M9's EnrollmentService issues real HTTP calls. Use `provideHttpClientTesting()` + `HttpTestingController` instead of mocking `HttpClient` directly.

Create `src/app/services/enrollment.service.spec.ts`:

```
describe("EnrollmentService", () => {
  let httpMock: HttpTestingController;
  let service: EnrollmentService;

  beforeEach(() => {
    TestBed.configureTestingModule({
      providers: [provideHttpClient(), provideHttpClientTesting()],
    });
    httpMock = TestBed.inject(HttpTestingController);
    service = TestBed.inject(EnrollmentService);
  });

  afterEach(() => httpMock.verify());

  it("getAll() issues GET /api/enrollments and maps the response",
    async () => {
      const result = firstValueFrom(service.getAll());

      const req = httpMock.expectOne((r) =>
        r.url.endsWith("/api/enrollments"));
      expect(req.request.method).toBe("GET");
      req.flush([
        { id: 1, studentId: 11, studentName: "Abeba", courseId: 101,
          courseName: "Intro to CS", status: "Pending", enrolledAt: "2026-08-12T10:00:00Z" },
        { id: 2, studentId: 12, studentName: "Kebede", courseId: 102,
          courseName: "Data Structures", status: "Approved", enrolledAt: "2026-08-12T10:05:00Z" },
      ]);

      const enrollments = await result;
      expect(enrollments).toHaveLength(2);
      expect(enrollments[0].courseName).toBe("Intro to CS");
    });

  it("approve(id) issues POST /api/enrollments/{id}/approve", async ()
    => {
      const result = firstValueFrom(service.approve(42));

      const req = httpMock.expectOne((r) =>
```

```

r.url.endsWith("/api/enrollments/42/approve"));
expect(req.request.method).toBe("PUT");
req.flush({
  id: 42,
  studentId: 11,
  studentName: "Abeba",
  courseId: 101,
  courseName: "Intro to CS",
  status: "Approved",
  enrolledAt: "2026-08-12T10:00:00Z",
});

const approved = await result;
expect(approved.status).toBe("Approved");
});
});

```

Run Frontend Tests

Run `npm test` and confirm both specs pass.

Verification Checkpoint (Exercise 4)

- ☐ Component spec sets signal inputs using `fixture.componentRef.setInput()`
- ☐ Spec waits for DOM stability using `await fixture.whenStable()`
- ☐ Output event subscription captures emitted course object
- ☐ EnrollmentStore spec verifies `entities()` length + first row's `courseName` after `store.seed(...)`
- ☐ EnrollmentStore spec verifies `pendingCount()` computed signal returns the right count
- ☐ EnrollmentService.`getAll()` spec asserts GET `/api/enrollments` against `HttpTestingController`
- ☐ EnrollmentService.`approve(id)` spec asserts PUT `/api/enrollments/{id}/approve`

Exercise 6: Playwright E2E

Unit and integration tests verify logic and the HTTP pipeline. This exercise verifies the **user facing surface** in a real browser, including semantic locators, auth reuse, and a failure-path story. This work subsumes the original retired Sessions 3 and 4.

Step 1: Install Playwright

```
npm init playwright@latest --legacy-peer-deps
```

Step 2: Auth Setup (reuse across tests)

The auth setup below navigates to /login, fills inputs by their accessible labels (Username, Password), clicks a button named Sign In, and waits for a heading whose name matches /command center/i.

```
// app.routes.ts (routes table that M9 left empty)
export const routes: Routes = [
  { path: "login", component: LoginComponent },
  { path: "command-center", component: InstructorDashboardComponent, canActivate: [authGuard] },
  { path: "**", redirectTo: "login" },
];

<!-- Login.component.html minimal shape labels and button name MUST match -->
<form (submit)="submit($event)">
  <label>Username <input name="username" /></label>
  <label>Password <input name="password" type="password" /></label>
  <button type="submit">Sign In</button>
</form>
```

The auth.setup heading regex /command center/i matches the `<h1>Instructor Command Center</h1>` that M9's `InstructorDashboardComponent` already renders. If your cohort customised the heading text, update the regex in `auth.setup.ts` (and the workbook / essentials copies) to match. Without a /login route, `page.goto("/login")` 404s and the setup hangs on `expect.toBeVisible`, but the auth API may still return 200, so the test is correctly catching a routing gap, not a flaky runner.

Create `e2e/auth.setup.ts`:

```
import { test as setup, expect } from "@playwright/test";

setup("authenticate as admin", async ({ page }) => {
  await page.goto("/login");
  // M10 & M12 baseline LoginRequest uses Email (with Username supporte
```

```

d as fallback).
    await page.getByLabel(/email|username/i).fill(process.env.TMS_ADMIN_EMAIL ?? process.env.TMS_ADMIN_USER!);
    await page.getByLabel("Password").fill(process.env.TMS_ADMIN_PASS!);
    await page.getByRole("button", { name: "Sign In" }).click();
    // M9's InstructorDashboardComponent renders <h1>Instructor Command Center</h1>; the
    // regex matches that heading so we know post-login navigation resolved the protected route.
    await expect(page.getByRole("heading", { name: /command center/i })).toBeVisible();

    await page.context().storageState({ path: "playwright/.auth/admin.json" });
});

```

Wire it up in playwright.config.ts:

```

projects: [
  { name: "setup", testMatch: /\.*\setup\.ts/ },
  {
    name: "tests",
    dependencies: ["setup"],
    use: { storageState: "playwright/.auth/admin.json" },
  },
],

```

Security rule: playwright/.auth/ contains session cookies. Confirm it is in .gitignore (it should already be from the pre-work checklist). Never commit storageState to version control.

Step 3: Happy-Path E2E Spec

Create e2e/admin-approve-enrollment.spec.ts:

```

import { test, expect } from "@playwright/test";

test("admin approves a pending enrollment", async ({ page }) => {
  await page.goto("/dashboard");

  // The dashboard heading text comes from M9's InstructorDashboardComponent template
  // ("Instructor Command Center"); the spec's regex matches that exactly so a future
  // copy edit doesn't silently break the auth-setup handoff.
  await expect(page.getByRole("heading", { name: /command center/i })).toBeVisible();

  // M9's EnrollmentListComponent renders a per-row "Approve" button on

```

```

ly when the
    // enrollment is still Pending. We click the first one and assert the
    optimistic
    // status flip from M9's EnrollmentStore shows up in the row's badge.
    const firstApprove = page.getByRole("button", { name: "Approve" }).first();
    await firstApprove.click();

    // The row's status badge flips to "Approved" instantly no navigation
    needed.
    await expect(page.getByText("Approved").first()).toBeVisible();
});

```

Run the E2E Suite

```
npx playwright test
```

Final QA Verification Checklist

Before completing Module 12 , run all test suites across the solution to ensure full safety net coverage:

1. .NET Backend Unit & Integration Tests:

```
dotnet test
```

2. Angular Vitest Suite:

```
npm test
```

3. Playwright E2E Suite:

```
npx playwright test
```

Verification Checkpoint (Exercise 6)

- ☐ Playwright installed and configured
- ☐ Setup project executes and saves storageState
- ☐ `auth.setup.ts` uses `getByLabel(/email|username/i)` (matching M10 LoginRequest)
- ☐ `auth.setup.ts` reads `TMS_ADMIN_USER` (or `TMS_ADMIN_EMAIL` alias) and waits for the `/command center/i` heading from M9's dashboard
- ☐ Happy-path E2E spec clicks the M9 Approve button (Pending row) and asserts the row flips to Approved